Dossier d'exploitation

Olympic Participation Tracker (Angular / Nginx) et Workshop Organizer (Spring Boot / PostgreSQL) — version 1.0 du 15/10/2026 — Bruno Hoarau

Sommaire

Sommaire 1
1. Guide d'installation 2
  1.1 Prérequis système 2
  1.2 Installation de l'application front-end (Olympic Participation Tracker) 3
  1.3 Installation de l'application back-end (Workshop Organizer API) 4
  1.4 Vérification de l'installation 5
  1.5 Troubleshooting 5
2. Plan de sauvegarde des données 7
  2.1 Données à sauvegarder 7
  2.2 Stratégie de sauvegarde 7
  2.3 Procédure de backup 8
  2.4 Procédure de restauration 9
  2.5 Tests de restauration 10
3. Surveillance (monitoring) 12
  3.1 Métriques à surveiller 12
  3.2 Seuils d'alerte 12
  3.3 Healthchecks 13
  3.4 Gestion des logs 14
  3.5 Alertes et escalation 15
Notes et références 16

1. Guide d'installation

Cette section permet à une personne tierce d'installer les deux applications sur un serveur Linux avec Docker Compose, puis de vérifier chaque étape. **Toutes les commandes ont été exécutées le 15/10/2026** sur un clone neuf du dépôt (Docker Engine 29.5.3, Docker Compose v5.1.4) ; les durées et sorties indiquées sont celles relevées pendant ces tests.

Service	Image / build	Port hôte	Conteneur
Front Olympic (Nginx)	`olympic-games-starter/Dockerfile` (Node 22 → `nginx:1.27-alpine`)	80	`olympic-games-starter`
API Workshop (Spring Boot)	`workshop-organizer/Dockerfile` (Gradle 8.7 / JDK 21 → `eclipse-temurin:21-jre`)	8080	`workshop-app`
Base PostgreSQL	`postgres:13`, volume `workshop-organizer_db-data`	5432 (127.0.0.1 seulement)	`workshop-db`

Les deux applications sont indépendantes : chacune a son propre `docker-compose.yml` et s'installe séparément. Répertoire d'installation retenu dans tout ce document : `/opt/p8`.

1.1 Prérequis système

Élément	Exigence	Justification / vérification
Système d'exploitation	Linux 64 bits : Ubuntu Server 24.04 LTS recommandé (toute distribution supportée par Docker Engine). Poste de développement : Windows 11 + Docker Desktop (WSL 2) ou macOS	Images Linux amd64. Vérifier : `cat /etc/os-release`
Docker Engine	≥ 24 (testé : 29.5.3)	`docker version --format '{{.Server.Version}}'`
Docker Compose (plugin v2)	≥ v2.20 (testé : v5.1.4)	Requis pour `depends_on: condition: service_healthy`, les `secrets` et les extensions `x-`. Vérifier : `docker compose version`
Git	≥ 2.30	`git --version`
Outils	`curl`, `openssl`, `tar`, `sha256sum` (paquets de base)	Génération du secret, vérifications, sauvegardes
Node.js / JDK	**Non requis sur le serveur** : la compilation a lieu dans les images (builds multi-étapes). Uniquement pour développer : Node 22, JDK 21 (Temurin)	Évite les écarts de version entre postes et serveur
Processeur / mémoire	2 vCPU, 4 Go de RAM minimum (build compris)	Mesuré en fonctionnement : API 357 Mo, PostgreSQL 37 Mo, Nginx 3 Mo ; le build Gradle de l'image demande ~2 Go
Disque	20 Go libres minimum, dont la place des sauvegardes	Images : API 569 Mo, `postgres:13` 624 Mo, front 74 Mo, plus le cache de build
Ports entrants	80 (front), 8080 (API)	5432 n'est publié que sur 127.0.0.1 (administration et sauvegardes locales)
Accès sortants	github.com, ghcr.io, registry-1.docker.io ; pour un build local : registry.npmjs.org, services.gradle.org, repo.maven.apache.org	Téléchargement du code, des images et des dépendances

Élément	Exigence	Justification / vérification
Droits	Utilisateur membre du groupe `docker`	`docker run --rm hello-world` doit fonctionner sans `sudo`

Installation de Docker (Ubuntu), si nécessaire

```
# dépôt officiel Docker (Engine + plugin Compose)
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER" && newgrp docker  # utiliser Docker sans sudo
docker version --format '{{.Server.Version}}' && docker compose version
```

Récupération du code (commun aux deux applications)

```
sudo mkdir -p /opt/p8 && sudo chown "$USER" /opt/p8
git clone \
 https://github.com/brunoHoarau/Hoarau_Bruno_08_Mettez_en_place_un_pipeline_CI-CD_simple_avec_Docker.git \
 /opt/p8
# vérification : dernier commit affiché
cd /opt/p8 && git log --oneline -1
```

Sous Windows, si le clone échoue avec `Filename too long`, activer d'abord les chemins longs : `git config --global core.longpaths true` (voir 1.5).

1.2 Installation de l'application front-end (Olympic Participation Tracker)

Aucune configuration n'est nécessaire : l'application est entièrement statique (fichiers Angular compilés et données `assets/mock/olympic.json` intégrés à l'image).

Étape 1 : contrôler la configuration

```
cd /opt/p8/olympic-games-starter
docker compose config --quiet && echo "configuration OK"
```

Étape 2 : construire et démarrer

```
docker compose up -d --build
```

Durée mesurée : 26 s avec le cache Docker, quelques minutes au premier build (téléchargement de Node et des dépendances npm).

Étape 3 : vérifier

```
# STATUS doit afficher (healthy) après ~10 s
docker compose ps
# attendu : 200
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/
# attendu : 200 (repli SPA)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/route/profonde
# attendu : [{"id":1,"country":"Italy"...
curl -s http://localhost/assets/mock/olympic.json | head -c 60; echo
```

Variante : installer l'image publiée par la CI (sans build)

Le pipeline publie chaque version sur GHCR (tags `<branche>-<sha>`, `<branche>`, `latest` sur `main`). Pour déployer exactement l'image testée par la CI :

```
REG=ghcr.io/brunohoarau/hoarau_bruno_08_mettez_en_place_un_pipeline_ci-cd_simple_avec_docker
# ou une version précise, ex. ci-docker-build-28b9346
TAG=main
docker pull "$REG/olympic-games-starter:$TAG"
```

```
docker tag  "$REG/olympic-games-starter:$TAG" olympic-games-starter:latest
docker compose up -d --no-build
```

1.3 Installation de l'application back-end (Workshop Organizer API)

L'API a besoin de deux fichiers **non versionnés** (ignorés par `.gitignore`) à créer à l'installation : `.env` (nom de la base et de l'utilisateur) et `secrets/db_password.txt` (mot de passe). Le mot de passe n'apparaît dans aucune variable d'environnement : il est monté comme secret Docker dans `/run/secrets/db_password`, lu par PostgreSQL (`POSTGRES_PASSWORD_FILE`) et par `entrypoint.sh` pour Spring Boot.

Étape 1 : créer le fichier .env

```
cd /opt/p8/workshop-organizer
cat > .env <<'EOF'
POSTGRES_DB=workshopsdb
POSTGRES_USER=workshops_user
EOF
```

Étape 2 : générer le secret PostgreSQL

```
mkdir -p secrets
( umask 077 && openssl rand -hex 24 | tr -d '\r\n' > secrets/db_password.txt )
wc -c < secrets/db_password.txt       # attendu : 48
# attendu : -rw------- (lisible par le seul propriétaire)
ls -l secrets/
```

Le `tr -d '\r\n'` évite qu'une fin de ligne (CRLF sous Windows) soit intégrée au mot de passe. **Conserver ce secret** : il est indispensable pour réutiliser le volume ou restaurer une sauvegarde de volume (voir 2.1). PostgreSQL ne l'applique qu'à la **première** initialisation du volume.

Étape 3 : contrôler la configuration

```
# aucun avertissement « variable is not set » ne doit apparaître
docker compose config --quiet && echo "configuration OK"
```

Étape 4 : construire et démarrer (PostgreSQL puis API)

```
docker compose up -d --build
```

Compose démarre d'abord `db` ; `app` n'est lancé qu'une fois PostgreSQL `healthy` (`depends_on: condition: service_healthy`). Le schéma (tables `notion_model`, `workshop_model`, `workshop_model_notions`) est créé automatiquement par Hibernate au premier démarrage (`spring.jpa.hibernate.ddl-auto=update`). Durée mesurée : 28 s jusqu'à `healthy` avec le cache ; le premier build compile l'application et exécute ses tests unitaires (plusieurs minutes).

Étape 5 : vérifier

```
docker compose ps                                       # db et app : (healthy)
# « Started ... in 12.9 seconds »
docker compose logs app | grep "Started JavaBasicAppApplication"
# « accepting connections »
docker exec workshop-db pg_isready -U workshops_user -d workshopsdb
# attendu : []  -> 200
curl -s -w "  -> %{http_code}\n" http://localhost:8080/api/notions
```

Variante : image publiée par la CI (sans build)

```
REG=ghcr.io/brunohoarau/hoarau_bruno_08_mettez_en_place_un_pipeline_ci-cd_simple_avec_docker
TAG=main
docker pull "$REG/workshop-organizer:$TAG"
docker tag  "$REG/workshop-organizer:$TAG" workshop-organizer:latest
```

```
docker compose up -d --no-build       # testé : API healthy en 27 s
```

1.4 Vérification de l'installation

Recette d'installation à dérouler après toute installation ou mise à jour. Résultats obtenus pendant les tests :

Interfaces et API

```
curl -s -o /dev/null -w "front %{http_code} %{time_total}s\n" http://localhost/
API=http://localhost:8080/api; JSON='Content-Type: application/json'
curl -s -X POST $API/notions -H "$JSON" -d '{"name":"Docker"}'
curl -s -X POST $API/notions -H "$JSON" -d '{"name":"PostgreSQL"}'
curl -s -X POST $API/workshops -H "$JSON" \
 -d '{"name":"Sauvegarder une base PostgreSQL",
      "notions":[{"name":"Docker"},{"name":"PostgreSQL"}]}'
curl -s $API/workshops
# attendu : [{"id":1,"name":"Sauvegarder une base PostgreSQL",
#             "notions":[{"name":"Docker"},{"name":"PostgreSQL"}]}]
```

Persistance dans PostgreSQL

```
docker exec workshop-db psql -U workshops_user -d workshopsdb -c '\dt' -c \
 "select (select count(*) from notion_model) notions,
         (select count(*) from workshop_model) ateliers,
         (select count(*) from workshop_model_notions) liens;"
# attendu : 3 tables ; notions = 2, ateliers = 1, liens = 2

# arrêt / redémarrage SANS -v : le volume est conservé
docker compose down && docker compose up -d
# l'atelier créé est toujours présent
curl -s http://localhost:8080/api/workshops
```

Logs et santé

```
docker compose ps --format 'table {{.Name}}\t{{.Status}}\t{{.Ports}}'
docker compose logs app | grep -c " ERROR "    # attendu : 0
# consommation de référence (voir 3.1)
docker stats --no-stream
```

Les données de recette (notions « Docker » et « PostgreSQL ») peuvent être conservées ou supprimées : `docker exec workshop-db psql -U workshops_user -d workshopsdb -c "truncate workshop_model_notions, workshop_model, notion_model"`.

1.5 Troubleshooting

Erreurs rencontrées ou reproduites pendant les tests, avec le message exact affiché.

Message / symptôme	Cause	Solution
`Bind for 0.0.0.0:80 failed: port is already allocated`	Port déjà utilisé (autre conteneur, Apache, IIS…)	Identifier l'occupant : `sudo ss -ltnp | grep ':80 '` (Windows : `netstat -ano | findstr :80`). L'arrêter, ou changer le port hôte dans `docker-compose.yml` (ex. `"8081:80"`).
`permission denied while trying to connect to the Docker daemon socket`	Utilisateur absent du groupe `docker`	`sudo usermod -aG docker $USER`, se reconnecter, vérifier avec `docker run --rm hello-world`.
`The "POSTGRES_DB" variable is not set. Defaulting to a blank string.`	`.env` absent, ou commande lancée hors de `workshop-organizer/`	Créer `.env` (1.3 étape 1) et lancer Compose depuis ce dossier ; contrôler avec `docker compose config`.

Message / symptôme	Cause	Solution
`dependency failed to start: container workshop-db is unhealthy` ; dans `docker logs workshop-db` : `Error: Database is uninitialized and superuser password is not specified.`	Fichier secret absent au premier lancement : Docker a créé à sa place un **répertoire** `secrets/db_password.txt`	`docker compose down -v`, `rm -rf secrets/db_password.txt`, recréer le fichier (1.3 étape 2), relancer.
`FATAL: password authentication failed for user "workshops_user"` dans les logs de `app`, qui redémarre en boucle	Le secret a changé alors que le volume a été initialisé avec l'ancien mot de passe	Remettre l'ancien secret (sauvegarde de configuration, 2.4 scénario B), ou changer le mot de passe en base : `docker exec -it workshop-db psql -U workshops_user -d workshopsdb`, puis `\password workshops_user` (saisie masquée) et même valeur dans `secrets/db_password.txt`.
`Erreur : ni secret /run/secrets/db_password ni variable SPRING_DATASOURCE_PASSWORD`	Conteneur API lancé hors Compose sans mot de passe	Utiliser `docker compose up`, ou fournir `SPRING_DATASOURCE_PASSWORD`.
`pull access denied for workshop-organizer, repository does not exist`	`--no-build` sans image locale portant le nom attendu	Construire (`docker compose up -d --build`) ou faire `docker pull` + `docker tag` (variante 1.3).
Front `unhealthy` alors que le site répond ; `docker inspect` : `wget: can't connect to remote host: Connection refused`	Ancien healthcheck sur `localhost` : résolu en `::1` (IPv6) alors que Nginx n'écoute qu'en IPv4	Corrigé dans `docker-compose.yml` (`http://127.0.0.1/`) ; appliquer avec `docker compose up -d`.
API `starting` puis `unhealthy` dans la première minute	Démarrage Spring Boot (~13 s mesurées) et connexion à la base	Attendre la fin du `start_period` (60 s) ; suivre `docker compose logs -f app` ; au-delà de 2 min, vérifier `pg_isready`.
Échec du build : délai dépassé sur `npm ci` ou téléchargement Gradle	Pas d'accès à registry.npmjs.org / services.gradle.org (proxy, pare-feu)	Configurer le proxy de Docker, relancer `docker compose build --no-cache`, ou utiliser la variante GHCR (sans build).
Build de l'API arrêté sur un test en échec	Le Dockerfile exécute `gradle clean test bootWar`	Lancer `./gradlew test` en local et consulter `build/reports/tests/test/index.html`.
Windows : `Filename too long` pendant le `git clone`	Limite de 260 caractères des chemins Windows	`git config --global core.longpaths true`, ou cloner dans un chemin court (`C:\dev`).
Windows (Git Bash) : chemin du type `C:/Program Files/Git/var/...` dans une commande `docker exec`	Conversion automatique des chemins par MSYS	Préfixer la commande par `MSYS_NO_PATHCONV=1`.
Windows : `400 Bad Request` sur un `POST` contenant des accents	Terminal non UTF-8 : JSON mal encodé	Envoyer le JSON depuis un fichier UTF-8 : `curl --data-binary @atelier.json`.

2. Plan de sauvegarde des données

Seule l'API Workshop possède des données persistantes (PostgreSQL). Le front est sans état : il se reconstruit à l'identique depuis git ou GHCR. Toutes les procédures ci-dessous ont été **exécutées et validées** le 15/10/2026 : sauvegarde, restauration après suppression accidentelle, restauration après perte du volume, restauration du volume et test de restauration automatisé.

Objectifs : **RPO 24 h** (perte de données maximale acceptée, dictée par la sauvegarde quotidienne) et **RTO 30 min** (remise en service). Mesuré : restauration de la base en 1 s, restauration + démarrage de l'API en 14 s.

2.1 Données à sauvegarder

Donnée	Emplacement	Criticité	Justification
Base `workshopsdb` (tables `notion_model`, `workshop_model`, `workshop_model_notions`, séquence `workshop_model_id_seq`)	Conteneur `workshop-db`, volume `workshop-organizer_db-data` (`/var/lib/postgresql/data`)	Critique	Seule donnée métier, non reconstructible.
Volume Docker `workshop-organizer_db-data` (fichiers PostgreSQL bruts)	`docker volume inspect workshop-organizer_db-data`	Élevée	Copie de secours physique : restauration complète rapide (même version majeure 13 obligatoire).
Configuration non versionnée : `workshop-organizer/.env`	Serveur, `/opt/p8/workshop-organizer`	Moyenne	Absente du dépôt git ; nécessaire pour redémarrer à l'identique.
Secrets : `workshop-organizer/secrets/db_password.txt`, clé de chiffrement des sauvegardes	Serveur ; coffre de l'équipe	Critique	Sans le mot de passe d'origine, une sauvegarde de volume est inutilisable (voir 1.5).
Configuration versionnée : `docker-compose.yml`, `nginx/nginx.conf`, `ops/*.sh`	Dépôt GitHub	Faible	Déjà protégée par git (historique + dépôt distant) ; archivée avec la configuration pour figer l'état déployé.
Images Docker	GHCR, tags `<branche>-<sha>`	Faible	Reconstructibles depuis git par la CI ; GHCR conserve chaque version.
Logs des conteneurs	`/var/lib/docker/containers/<id>/<id>-json.log`	Faible	Rotation à 3 × 10 Mo ; non sauvegardés (à centraliser si une conservation est exigée, voir 3.4).

2.2 Stratégie de sauvegarde

Donnée	Type	Fréquence	Rétention	Stockage
Base (`pg_dump -Fc`)	Complète, logique, à chaud	Quotidienne à 02:00 + avant chaque mise à jour	14 quotidiennes, 8 hebdomadaires, 12 mensuelles	Local `/var/backups/workshop-organizer` + copie hors site
Volume `db-data` (archive tar)	Complète, physique, à froid	Mensuelle + avant une montée de version de PostgreSQL	3 dernières	Hors site
Configuration + secrets (archive chiffrée AES-256)	Complète	À chaque modification + hebdomadaire	12 versions	Hors site + coffre de l'équipe

Donnée	Type	Fréquence	Rétention	Stockage
Images Docker	Gérée par GHCR	À chaque build CI	20 dernières versions de `main`	GHCR

- **Règle 3-2-1** : 3 copies (production, disque local de sauvegarde, stockage hors site), 2 supports, 1 copie hors site, chiffrée.
- **Pourquoi un dump logique en quotidien** : cohérent sans arrêt du service (instantané MVCC de `pg_dump`), indépendant de la version de PostgreSQL et très compact (8 Ko pour la base de recette) ; il permet aussi de restaurer une seule table.
- **Pourquoi une archive de volume en complément** : restauration à l'identique la plus simple après perte du serveur ; elle impose l'arrêt de la base (copie à froid) et la même version majeure, d'où une fréquence mensuelle.
- **Évolution** : si un RPO inférieur à 24 h est exigé, ajouter l'archivage continu des WAL (PITR) avec pgBackRest ou `archive_command`, pour une perte limitée à quelques minutes.

2.3 Procédure de backup

Sauvegarde de la base : script ops/backup-db.sh (versionné dans le dépôt ; extrait sans les commentaires)

```
#!/usr/bin/env bash
set -euo pipefail
BACKUP_DIR="${BACKUP_DIR:-/var/backups/workshop-organizer}"
RETENTION_DAYS="${RETENTION_DAYS:-14}"
DB_CONTAINER="${DB_CONTAINER:-workshop-db}"
set -a; . ./.env; set +a
ts="$(date +%Y%m%d-%H%M%S)"
file="$BACKUP_DIR/workshopsdb-$ts.dump"
mkdir -p "$BACKUP_DIR"
docker exec "$DB_CONTAINER" \
 pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc > "$file.tmp"
mv "$file.tmp" "$file"
docker exec -i "$DB_CONTAINER" pg_restore --list < "$file" > /dev/null
(cd "$BACKUP_DIR" && \
 sha256sum "$(basename "$file")" > "$(basename "$file").sha256")
find "$BACKUP_DIR" -name 'workshopsdb-*.dump*' -mtime +"$RETENTION_DAYS" -delete
echo "Sauvegarde OK : $file ($(du -h "$file" | cut -f1))"
```

Le dump est produit dans le conteneur par connexion locale (aucun mot de passe ne transite), écrit dans un fichier temporaire puis renommé (pas de fichier partiel en cas d'échec), contrôlé par `pg_restore --list`, puis accompagné de son empreinte SHA-256.

Exécution et vérification

```
sudo mkdir -p /var/backups/workshop-organizer
sudo chown "$USER" /var/backups/workshop-organizer
cd /opt/p8/workshop-organizer && ../ops/backup-db.sh
# Sauvegarde OK : /var/backups/.../workshopsdb-20261015-124324.dump (8,0K)
cd /var/backups/workshop-organizer
sha256sum -c workshopsdb-20261015-124324.dump.sha256     # ... : OK
docker exec -i workshop-db pg_restore --list \
 < workshopsdb-20261015-124324.dump | grep "TABLE DATA"
# 3 lignes : notion_model, workshop_model, workshop_model_notions
```

Planification (crontab de l'utilisateur d'exploitation : crontab -e)

```
B=/var/backups/workshop-organizer
W=/opt/p8/workshop-organizer
# Quotidienne 02:00
0 2 * * *  cd $W && ../ops/backup-db.sh >> /var/log/workshop-backup.log 2>&1
# Copie hebdomadaire (dimanche 03:00) et mensuelle (le 1er, 03:30) du dernier dump
0 3 * * 0  mkdir -p $B/weekly  && cp "$(ls -t $B/*.dump | head -1)" $B/weekly/
```

```
30 3 1 * * mkdir -p $B/monthly && cp "$(ls -t $B/*.dump | head -1)" $B/monthly/
# Rétention : hebdomadaire 8 semaines, mensuelle 12 mois
0 4 * * 0  find $B/weekly  -name '*.dump' -mtime +56  -delete
0 4 1 * *  find $B/monthly -name '*.dump' -mtime +365 -delete
```

Configuration et secrets (archive chiffrée)

```
# Clé de chiffrement : créée une fois, copiée dans le coffre de l'équipe
( umask 077 && openssl rand -base64 32 | tr -d '\r\n' > /root/.workshop-backup.key )

cd /opt/p8/workshop-organizer
B=/var/backups/workshop-organizer; KEY=file:/root/.workshop-backup.key
tar czf - .env docker-compose.yml secrets/ \
 | openssl enc -aes-256-cbc -pbkdf2 -salt -pass $KEY \
   -out $B/config-$(date +%Y%m%d-%H%M%S).tar.gz.enc
# Vérification : liste du contenu sans rien extraire
openssl enc -d -aes-256-cbc -pbkdf2 -pass $KEY \
 -in $B/config-<horodatage>.tar.gz.enc | tar tzf -
# attendu : .env  docker-compose.yml  secrets/  secrets/db_password.txt
```

Volume Docker (copie à froid, mensuelle)

```
cd /opt/p8/workshop-organizer
# arrêt de l'API et de la base : copie cohérente
docker compose stop
docker run --rm -v workshop-organizer_db-data:/data:ro \
 -v /var/backups/workshop-organizer:/backup \
 alpine:3.20 tar czf /backup/db-data-$(date +%Y%m%d-%H%M%S).tar.gz -C /data .
docker compose start
```

Interruption de service mesurée : moins de 30 s. Taille de l'archive de recette : 5,7 Mo.

Copie hors site

Après chaque sauvegarde, synchroniser le dossier vers un stockage objet chiffré situé dans un autre datacenter, par exemple `rclone sync /var/backups/workshop-organizer remote:workshop-backups`. Commande donnée à titre d'exemple : elle dépend du fournisseur retenu et n'a pas été exécutée dans ce projet. Vérifier ensuite les empreintes avec `sha256sum -c`.

2.4 Procédure de restauration

Avant toute restauration, faire une sauvegarde de précaution de l'état actuel (`../ops/backup-db.sh`) et vérifier l'empreinte du fichier à restaurer (`sha256sum -c <fichier>.sha256`).

Scénario A : erreur de manipulation ou données corrompues (volume intact)

```
cd /opt/p8/workshop-organizer
DUMP=/var/backups/workshop-organizer/workshopsdb-20261015-124324.dump
# plus aucune écriture pendant la restauration
docker compose stop app
docker exec -i workshop-db pg_restore -U workshops_user -d workshopsdb \
 --clean --if-exists --no-owner --single-transaction < "$DUMP"
docker compose start app
```

`--clean --if-exists` supprime puis recrée chaque objet ; `--single-transaction` garantit tout ou rien. Testé : après suppression de tous les ateliers (0 atelier, 0 lien), retour à 2 notions, 1 atelier, 2 liens et API `healthy`.

Scénario B : perte du volume ou nouveau serveur

```
# 1. Réinstaller le code (1.1), puis restaurer .env et le secret
#    depuis l'archive chiffrée
cd /opt/p8/workshop-organizer
openssl enc -d -aes-256-cbc -pbkdf2 -pass file:/root/.workshop-backup.key \
```

```
 -in /var/backups/workshop-organizer/config-<horodatage>.tar.gz.enc \
 | tar xzf - .env secrets/
# 2. Démarrer la base seule (vide) et attendre qu'elle accepte les connexions
docker compose up -d db
until docker exec workshop-db pg_isready -q -U workshops_user -d workshopsdb
do sleep 2; done
# 3. Restaurer le dump, puis démarrer l'API
docker exec -i workshop-db pg_restore -U workshops_user -d workshopsdb \
 --no-owner --single-transaction < "$DUMP"
docker compose up -d app
```

Testé après `docker compose down -v` (volume supprimé) : restauration en 1 s, contenu complet, API `healthy`. La séquence est restaurée : l'atelier créé ensuite reçoit l'identifiant 2, sans collision.

Scénario C : restauration de l'archive du volume

```
cd /opt/p8/workshop-organizer
# ATTENTION : supprime le volume actuel
docker compose down -v
# recrée le volume vide, géré par Compose
docker compose up --no-start
docker run --rm -v workshop-organizer_db-data:/data \
 -v /var/backups/workshop-organizer:/backup:ro \
 alpine:3.20 tar xzf /backup/db-data-<horodatage>.tar.gz -C /data
docker compose up -d
```

Prérequis : le secret `db_password.txt` d'origine (les comptes sont dans les fichiers restaurés) et la même version majeure de PostgreSQL (13). Créer le volume avec `up --no-start` évite l'avertissement `volume ... already exists but was not created by Docker Compose`.

Vérification après chaque restauration

```
# db et app (healthy)
docker compose ps
docker exec workshop-db psql -U workshops_user -d workshopsdb -tA \
 -c "select count(*) from notion_model" -c "select count(*) from workshop_model" \
 -c "select count(*) from workshop_model_notions"
curl -s -w "  -> %{http_code}\n" http://localhost:8080/api/workshops
```

2.5 Tests de restauration

- **Mensuel** (premier lundi du mois) : test automatisé `ops/restore-test.sh` sur la dernière sauvegarde.
- **Trimestriel** : exercice complet du scénario B sur une machine distincte, déchiffrement de la configuration compris, chronométré pour vérifier le RTO.
- **Après tout changement** de procédure, de version de PostgreSQL ou de schéma.

Test automatisé : ops/restore-test.sh

Le script vérifie l'empreinte de la dernière sauvegarde, la restaure dans un PostgreSQL 13 jetable (`rt-db`), démarre l'image de l'API dessus (`rt-app`), compare le contenu à la production puis supprime tout ce qu'il a créé (`trap`). **La production n'est jamais modifiée.**

```
cd /opt/p8/workshop-organizer
../ops/restore-test.sh | tee -a /var/log/workshop-restore-tests.log
```

Résultat du test exécuté le 15/10/2026 :

```
Sauvegarde testée : workshopsdb-20261015-124716.dump
Restaurée         : 2 notions, 1 ateliers, 2 liens
Production        : 2 notions, 1 ateliers, 2 liens
API restaurée     : [{"id":1,"name":"Sauvegarder une base PostgreSQL","notions":[...]}]
Durée (restauration + démarrage API) : 14 s
RESULTAT : SUCCES
```

Critères de succès

Critère	Attendu
Intégrité du fichier	`sha256sum -c` : OK
Restauration	`pg_restore --exit-on-error` sans erreur (code retour 0)
Contenu	Mêmes nombres de notions, ateliers et liens que la production au moment de la sauvegarde
Fonctionnement	L'API démarre sur la base restaurée et `GET /api/workshops` renvoie les données (HTTP 200)
Durée	< 5 min pour le test automatisé ; < 30 min (RTO) pour l'exercice complet

Chaque test est consigné (date, sauvegarde testée, durée, résultat, opérateur) dans `/var/log/workshop-restore-tests.log` et dans le ticket mensuel d'exploitation. Un échec déclenche une alerte critique (3.2) : la sauvegarde n'est plus considérée comme fiable tant que la cause n'est pas corrigée.

3. Surveillance (monitoring)

En place aujourd'hui : healthchecks Docker des trois services (3.3), redémarrage automatique (`restart: unless-stopped`), rotation des logs (3.4) et commandes Docker de diagnostic. **Recommandé pour la production** : une pile Prometheus + Grafana + Alertmanager, standard open source adapté à Docker, qui collecte les métriques de l'hôte (node_exporter), des conteneurs (cAdvisor), de PostgreSQL (postgres_exporter), de l'API (Spring Boot Actuator + Micrometer) et des sondes HTTP (Blackbox exporter).

3.1 Métriques à surveiller

Couche	Métriques	Source / commande	Valeur de référence mesurée
Hôte	CPU, mémoire, espace disque (`/var/lib/docker`, `/var/backups`), charge, réseau	node_exporter ; `df -h`, `free -m`, `uptime`	—
Conteneurs	CPU %, mémoire utilisée, redémarrages, état de santé, E/S réseau et disque	cAdvisor ; `docker stats --no-stream`, `docker inspect`	API 357 Mo / 0,5 % CPU ; base 37 Mo ; Nginx 3 Mo ; 0 redémarrage
API Workshop	Disponibilité, temps de réponse (p95), requêtes/s, taux de 5xx, tas JVM, GC, pool Hikari (actives / en attente)	Actuator + Micrometer (`/actuator/prometheus`) ; `curl -w '%{time_total}'`	`/api/workshops` 22 ms, `/api/notions` 16 ms
PostgreSQL	Disponibilité, connexions / `max_connections`, taille de la base, transactions/s, verrous, requêtes longues	postgres_exporter ; `pg_isready`, `pg_stat_activity`, `pg_database_size()`	11 connexions / 100 (pool Hikari de 10) ; base 8 Mo
Front Nginx	Disponibilité, temps de réponse, taux de 4xx / 5xx	Blackbox exporter ; logs d'accès Nginx	`/` 8 ms
Sauvegardes	Âge et taille du dernier dump, résultat du dernier test de restauration	Script de contrôle (`find -mmin`) ou métrique poussée vers Prometheus	Dump 8 Ko ; test de restauration réussi en 14 s

Les seuils de temps de réponse sont alignés sur ceux du plan de tests (p95 < 300 ms en lecture) pour qu'un même objectif soit vérifié avant la mise en production et surveillé ensuite.

3.2 Seuils d'alerte

Métrique	Warning	Critical	Action
CPU hôte	> 70 % pendant 10 min	> 90 % pendant 5 min	`docker stats` : identifier le conteneur ; redimensionner
Mémoire hôte	> 80 %	> 90 %	Vérifier les fuites (tas JVM) ; augmenter la RAM
Disque libre (`/var/lib/docker`, `/var/backups`)	< 20 %	< 10 %	`docker system df`, `docker image prune` ; purger les anciennes sauvegardes locales
Redémarrages d'un conteneur	≥ 1 par heure	≥ 3 en 15 min (boucle)	`docker logs --tail 100 <conteneur>` ; voir 1.5
Healthcheck	Un échec (statut `starting` prolongé)	`unhealthy` (3 échecs consécutifs)	Diagnostiquer avec `docker inspect` ; redémarrer le service
Disponibilité HTTP (front, API)	1 sonde en échec	3 sondes consécutives (~1 min 30)	Astreinte immédiate (3.5)
Temps de réponse API p95	> 300 ms pendant 5 min	> 1 s pendant 5 min	Requêtes lentes PostgreSQL, saturation du pool, GC

Métrique	Warning	Critical	Action
Taux d'erreurs 5xx	> 1 % sur 5 min	> 5 % sur 5 min	Logs de l'API ; retour à l'image précédente si régression
Tas JVM après GC	> 80 %	> 90 %	Analyser la mémoire ; ajuster `-Xmx` / la limite du conteneur
Pool Hikari	≥ 8 connexions actives sur 10	Requêtes en attente > 0 pendant 1 min	Requêtes longues ; taille du pool
Connexions PostgreSQL	> 70 / 100	> 90 / 100	`select * from pg_stat_activity` ; fuite de connexions
Requête PostgreSQL active	> 30 s	> 5 min	`pg_cancel_backend(pid)` après analyse
Âge de la dernière sauvegarde	> 26 h	> 50 h ou échec du script	Relancer `ops/backup-db.sh` ; consulter `/var/log/workshop-backup.log`
Test de restauration	Non exécuté depuis 35 jours	En échec	Bloquer les mises à jour tant que les sauvegardes ne sont pas fiables

Les seuils mémoire des conteneurs supposent des limites explicites (recommandées : API 1 Go, base 512 Mo, Nginx 64 Mo, soit environ trois fois la consommation mesurée) via `mem_limit` dans `docker-compose.yml`.

3.3 Healthchecks

Healthchecks configurés dans docker-compose.yml

Service	Commande	Intervalle / délai / essais	Démarrage
`olympic-games-starter`	`wget -qO- http://127.0.0.1/`	30 s / 5 s / 3	10 s
`workshop-db`	`pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}`	5 s / 5 s / 10	10 s
`workshop-app`	`curl -fsS -o /dev/null http://127.0.0.1:8080/api/notions`	30 s / 5 s / 3	60 s

- **Usage au démarrage** : `app` attend que `db` soit `healthy` (`depends_on: condition: service_healthy`).
- **Usage en exploitation** : l'état est visible dans `docker compose ps` et exploitable par la supervision. Attention : Docker **ne redémarre pas** un conteneur `unhealthy` ; `restart: unless-stopped` ne réagit qu'à l'arrêt du processus. L'état `unhealthy` doit donc déclencher une alerte (3.2).
- **Correction apportée** : le healthcheck du front visait `localhost`, résolu en `::1` (IPv6) dans l'image, alors que Nginx n'écoute qu'en IPv4 : le conteneur était `unhealthy` en permanence. Il vise désormais `127.0.0.1`.

Commandes de vérification

```
# état de chaque service
docker compose ps
docker inspect -f '{{.State.Health.Status}} restarts={{.RestartCount}}' workshop-app
# 5 derniers résultats du healthcheck
docker inspect -f '{{json .State.Health.Log}}' olympic-games-starter
# /var/run/postgresql:5432 - accepting connections
docker exec workshop-db pg_isready -U workshops_user -d workshopsdb
F='%{http_code} %{time_total}s\n'
curl -fsS -o /dev/null -w "$F" http://localhost:8080/api/notions
curl -fsS -o /dev/null -w "$F" http://localhost/
docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'
```

Recommandation : endpoint de santé dédié (Spring Boot Actuator)

Le healthcheck de l'API appelle un endpoint métier (`/api/notions`), dont le coût augmente avec le volume de données. Ajouter `spring-boot-starter-actuator`, puis exposer `management.endpoints.web.exposure.include=health,prometheus` et `management.endpoint.health.probes.enabled=true` : `/actuator/health` (avec contrôle de la base), `/actuator/health/liveness` et `/actuator/health/readiness` serviront aux healthchecks Docker et aux sondes Kubernetes, et `/actuator/prometheus` à la collecte des métriques. Le préfixe `/api` de `WebMvcConfig` ne s'applique qu'aux `@RestController` : les URL d'Actuator ne sont pas concernées.

3.4 Gestion des logs

Service	Contenu et niveau	Emplacement	Rotation / conservation
Front (Nginx)	Accès (format `main`) et erreurs (niveau `warn`), écrits sur stdout / stderr du conteneur	`docker logs olympic-games-starter` ; fichier `/var/lib/docker/containers/<id>/<id>-json.log`	json-file : 3 × 10 Mo
API (Spring Boot)	Console, niveau `INFO` par défaut ; requêtes SQL affichées (`spring.jpa.show-sql=true`, lignes `Hibernate:`)	`docker logs workshop-app`	json-file : 3 × 10 Mo
PostgreSQL	stderr, niveau par défaut (connexions refusées, erreurs, checkpoints)	`docker logs workshop-db`	json-file : 3 × 10 Mo
Sauvegardes et tests de restauration	Sortie des scripts `ops/`	`/var/log/workshop-backup.log`, `/var/log/workshop-restore-tests.log`	logrotate : hebdomadaire, 8 fichiers
CI/CD	Logs des workflows, rapports de tests	GitHub Actions	Logs : 90 jours ; artefacts : 14 jours

Consultation

```
# temps réel (depuis workshop-organizer/)
docker compose logs -f app
docker logs --since 1h --tail 200 workshop-app      # dernière heure
docker logs workshop-app 2>&1 | grep -E " (ERROR|WARN) "
docker inspect -f '{{.LogPath}}' workshop-app       # fichier sur l'hôte
# {"Type":"json-file","Config":{"max-file":"3","max-size":"10m"}}
docker inspect -f '{{json .HostConfig.LogConfig}}' workshop-app
```

Configuration

- **Rotation** : définie dans chaque `docker-compose.yml` (`logging: driver json-file, max-size 10m, max-file 3`), soit 30 Mo maximum par conteneur. Sans cette option, le driver json-file n'a aucune limite et peut remplir le disque. Valeur par défaut possible pour tous les conteneurs dans `/etc/docker/daemon.json` (`"log-opts": {"max-size": "10m", "max-file": "3"}`).
- **Niveau de l'API** : modifiable sans rebuild par variable d'environnement dans `docker-compose.yml`, par exemple `LOGGING_LEVEL_ROOT: WARN` ou `LOGGING_LEVEL_FR_OC_DEVOPS: DEBUG` pour un diagnostic. En production, désactiver les requêtes SQL (`SPRING_JPA_SHOW_SQL: "false"`) : volumineuses et susceptibles d'exposer des données.
- **Requêtes lentes PostgreSQL** : ajouter `command: postgres -c log_min_duration_statement=500` pour journaliser les requêtes de plus de 500 ms.
- **Bruit** : les healthchecks produisent une ligne d'accès Nginx toutes les 30 s (`127.0.0.1 ... "Wget"`), à filtrer dans les analyses.

- **Centralisation recommandée** : Grafana Loki + Promtail (ou ELK) pour conserver 30 jours de logs, les corréler avec les métriques et alerter sur les motifs d'erreur.

3.5 Alertes et escalation

Criticité	Canaux	Destinataires
Critical	Appel / SMS par l'outil d'astreinte (PagerDuty ou Opsgenie), canal Slack `#p8-incidents`, e-mail	Ops d'astreinte, puis escalade (tableau ci-dessous)
Warning	Canal Slack `#p8-alertes`, e-mail	Équipe Ops et Dev référent de l'application, en heures ouvrées
Info	Tableau de bord Grafana uniquement	Consultation libre

Le routage est réalisé par Alertmanager à partir du label `severity` des règles Prometheus ; une alerte est regroupée par service pour éviter les notifications en rafale.

Procédure d'escalation

Niveau	Prise en charge (Critical)	Prise en charge (Warning)	Rôle
N1 : Ops d'astreinte	15 min	4 h ouvrées	Acquitter l'alerte, diagnostiquer (1.5, 3.3, 3.4), redémarrer, restaurer (2.4), revenir à l'image précédente
N2 : Dev référent (front ou API)	Si non résolu en 30 min	1 jour ouvré	Analyse applicative, correctif, nouvelle version via la CI
N3 : Responsable technique	Si non résolu en 1 h	—	Décisions : communication aux utilisateurs, restauration avec perte de données, arbitrage
DevSecOps	Immédiat pour un incident de sécurité	—	Révocation et rotation des secrets, analyse de l'exposition

- Une alerte non acquittée dans le délai passe automatiquement au niveau suivant.
- Tout incident Critical donne lieu à un ticket et à un post-mortem sous 5 jours ouvrés (cause, chronologie, actions préventives).
- Retour arrière vers la version précédente, en général la solution la plus rapide :

```
REG=ghcr.io/brunohoarau/hoarau_bruno_08_mettez_en_place_un_pipeline_ci-cd_simple_avec_docker
docker pull "$REG/workshop-organizer:main-<sha précédent>"
docker tag  "$REG/workshop-organizer:main-<sha précédent>" workshop-organizer:latest
cd /opt/p8/workshop-organizer && docker compose up -d --no-build app
```

Notes et références

Tests réalisés pour ce dossier (15/10/2026)

Procédure	Résultat	Durée mesurée
Clone neuf + installation du front (build)	`healthy`, HTTP 200, repli SPA OK	26 s
Installation de l'API + PostgreSQL (build)	`healthy`, API 200, 3 tables créées	28 s
Installation de l'API depuis GHCR (`--no-build`)	`healthy`, API 200	27 s
Persistance après `down` / `up`	Données conservées	—
Sauvegarde `ops/backup-db.sh`	Dump 8 Ko, contrôle `pg_restore --list` et empreinte OK	< 1 s
Sauvegarde chiffrée de la configuration, archive du volume	Contenu vérifié	< 30 s d'arrêt
Restauration A (suppression accidentelle)	Données revenues, API `healthy`	1 s
Restauration B (volume supprimé)	Données et séquence restaurées	1 s
Restauration C (archive du volume)	État de la sauvegarde retrouvé	< 1 min
Test automatisé `ops/restore-test.sh`	SUCCES	14 s

Environnement de test : Windows 11 + Docker Desktop (Docker Engine 29.5.3, Compose v5.1.4), Git Bash. Les tests ont utilisé des noms de conteneurs et des ports distincts pour ne pas perturber l'installation existante ; les commandes de ce dossier utilisent les noms réels. Les particularités Windows rencontrées sont documentées en 1.5.

Modifications apportées au dépôt pour ce dossier

- `olympic-games-starter/docker-compose.yml` : healthcheck sur `127.0.0.1` (correction), rotation des logs.
- `workshop-organizer/docker-compose.yml` : healthcheck de l'API, `restart: unless-stopped`, rotation des logs, port PostgreSQL limité à `127.0.0.1`, nom d'image `workshop-organizer:latest` (variante GHCR).
- `ops/backup-db.sh` et `ops/restore-test.sh` : scripts de sauvegarde et de test de restauration.

Cohérence des versions

PostgreSQL 13 (compose, CI, tests de restauration) ; Java 21 / Spring Boot 3.2.4 ; Node 22 / Angular 20 ; Nginx 1.27 ; Docker Compose ≥ v2.20. Toute montée de version de PostgreSQL impose une sauvegarde logique préalable : une archive de volume n'est pas compatible d'une version majeure à l'autre.

Documentation officielle

- Docker Engine, installation sur Ubuntu : https://docs.docker.com/engine/install/ubuntu/
- Docker, utilisation sans sudo : https://docs.docker.com/engine/install/linux-postinstall/
- Compose, healthcheck, depends_on et secrets : https://docs.docker.com/reference/compose-file/services/
- Docker, sauvegarde et restauration des volumes : https://docs.docker.com/engine/storage/volumes/#back-up-restore-or-migrate-data-volumes
- Docker, driver de logs json-file : https://docs.docker.com/engine/logging/drivers/json-file/
- Image officielle PostgreSQL (POSTGRES_PASSWORD_FILE, initialisation) :

https://hub.docker.com/_/postgres

- PostgreSQL 13, pg_dump : https://www.postgresql.org/docs/13/app-pgdump.html
- PostgreSQL 13, pg_restore : https://www.postgresql.org/docs/13/app-pgrestore.html
- PostgreSQL 13, sauvegarde et restauration (PITR) : https://www.postgresql.org/docs/13/backup.html
- Spring Boot, Actuator et endpoints de santé : https://docs.spring.io/spring-boot/reference/actuator/endpoints.html
- Spring Boot, configuration des logs : https://docs.spring.io/spring-boot/reference/features/logging.html
- Prometheus / Alertmanager : https://prometheus.io/docs/alerting/latest/alertmanager/
- cAdvisor : https://github.com/google/cadvisor ; postgres_exporter : https://github.com/prometheus-community/postgres_exporter
- Règle de sauvegarde 3-2-1 (CISA) : https://www.cisa.gov/sites/default/files/publications/data_backup_options.pdf